Python Programming for Beginners

Part 3

Python Operators

Topics Covered

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Identity Operators
- Membership Operators
- Bitwise Operators

Learning Objectives

By the end of this lesson, you will be able to:

- Understand what operators are.
- Explain why operators are used.
- Use arithmetic operators.
- Use assignment operators.
- Compare values using comparison operators.
- Apply logical operators.
- Understand identity operators.
- Use membership operators.
- Learn the basics of bitwise operators.
- Write programs using operators.

What are Operators?

An **operator** is a special symbol or keyword that performs an operation on one or more values (operands).

Operators are used to perform tasks such as:

- Mathematical calculations
- Assigning values
- Comparing values

- Combining conditions
- Checking membership
- Checking object identity

Example

```
a = 10
b = 5

print(a + b)
```

Output

```
15
```

In this example:

- `+` is the operator.
- `a` and `b` are operands.

Why Do We Need Operators?

Operators help us perform different operations on data.

They are used to:

- Perform calculations.
- Compare values.
- Assign values to variables.
- Make decisions.
- Work with collections.
- Write efficient programs.

Without operators, programming would be difficult and repetitive.

Types of Operators

Python provides the following types of operators:

Operator Type	Purpose
Arithmetic	Perform mathematical operations
Assignment	Assign values
Comparison	Compare two values
Logical	Combine conditions
Identity	Compare object identity

Operator Type	Purpose
Membership	Check membership
Bitwise	Perform binary operations

Arithmetic Operators

Arithmetic operators are used to perform mathematical operations on numbers. They are the most commonly used operators in Python.

Arithmetic Operators Table

Operator	Name	Description	Example
`+`	Addition	Adds two values	`10 + 5`
`-`	Subtraction	Subtracts one value from another	`10 - 5`
`*`	Multiplication	Multiplies two values	`10 * 5`
`/`	Division	Divides one value by another	`10 / 5`
`%`	Modulus	Returns the remainder	`10 % 3`
`**`	Exponentiation	Raises a number to a power	`2 ** 3`
`//`	Floor Division	Returns the quotient without the decimal part	`10 // 3`

Example

In [ ]:
```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
print("Floor Division:", a // b)
```

Output

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Modulus: 1
Exponentiation: 1000
Floor Division: 3
```

Key Points

- Arithmetic operators are used to perform mathematical calculations.
- The `/` operator always returns a floating-point (`float`) result.
- The `%` operator returns the remainder after division.
- The `**` operator calculates the power of a number.
- The `//` operator removes the decimal part and returns the integer quotient.

Assignment Operators

Assignment operators are used to assign values to variables. They can also combine an arithmetic operation with assignment, making the code shorter and easier to read.

Assignment Operators Table

Operator	Description	Example	Equivalent To
`=`	Assign a value	`x = 10`	`x = 10`
`+=`	Add and assign	`x += 5`	`x = x + 5`
`-=`	Subtract and assign	`x -= 5`	`x = x - 5`
`*=`	Multiply and assign	`x *= 5`	`x = x * 5`
`/=`	Divide and assign	`x /= 5`	`x = x / 5`
`%=`	Modulus and assign	`x %= 5`	`x = x % 5`
`**=`	Exponentiate and assign	`x **= 2`	`x = x ** 2`
`//=`	Floor divide and assign	`x //= 5`	`x = x // 5`

Example

In [ ]:
```
x = 10

x += 5
print("After += :", x)

x -= 3
print("After -= :", x)

x *= 2
print("After *= :", x)

x /= 4
print("After /= :", x)

x %= 3
```

```
print("After %= :", x)

x = 5
x **= 2
print("After **= :", x)

x = 10
x //= 3
print("After //= :", x)
```

Output

```
After += : 15
After -= : 12
After *= : 24
After /= : 6.0
After %= : 0.0
After **= : 25
After //= : 3
```

Key Points

- The `=` operator assigns a value to a variable.
- Compound assignment operators perform an operation and assign the result in one step.
- Assignment operators make code shorter and easier to read.
- They are commonly used in loops, calculations, and data processing.

Comparison Operators

Comparison operators are used to compare two values. The result of a comparison is always a **Boolean** value:

- `True`
- `False`

These operators are commonly used in decision-making and conditional statements.

Comparison Operators Table

Operator	Name	Description	Example
`==`	Equal to	Returns `True` if both values are equal	`10 == 10`
`!=`	Not equal to	Returns `True` if values are different	`10 != 5`
`>`	Greater than	Returns `True` if the left value is greater	`10 > 5`
`<`	Less than	Returns `True` if the left value is smaller	`5 < 10`

Operator	Name	Description	Example
`>=`	Greater than or equal to	Returns `True` if the left value is greater than or equal to the right value	`10 >= 10`
`<=`	Less than or equal to	Returns `True` if the left value is less than or equal to the right value	`5 <= 10`

Example

In [ ]:
```
a = 10
b = 5

print("a == b :", a == b)
print("a != b :", a != b)
print("a > b  :", a > b)
print("a < b  :", a < b)
print("a >= b :", a >= b)
print("a <= b :", a <= b)
```

Output

```
a == b : False
a != b : True
a > b  : True
a < b  : False
a >= b : True
a <= b : False
```

Key Points

- Comparison operators compare two values.
- The result is always `True` or `False`.
- They are commonly used with `if`, `elif`, `else`, and loops.
- Use `==` to compare values, not `=`. The `=` operator is used for assignment.

Logical Operators

Logical operators are used to combine two or more conditions. They return a **Boolean** value (`True` or `False`) based on the result of the conditions.

Python provides three logical operators:

- `and`
- `or`
- `not`

Logical Operators Table

Operator	Description	Example
`and`	Returns `True` if **both** conditions are `True`	`a > 5 and b < 10`
`or`	Returns `True` if **at least one** condition is `True`	`a > 5 or b < 10`
`not`	Reverses the result of a condition	`not(a > 5)`

Example

In [ ]:
```
a = 10
b = 5

print("and :", a > 5 and b < 10)
print("or  :", a < 5 or b < 10)
print("not :", not(a > 5))
```

Output

```
and : True
or  : True
not : False
```

Truth Table

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Key Points

- Logical operators combine one or more conditions.
- They always return `True` or `False`.
- `and` returns `True` only if **all** conditions are `True`.
- `or` returns `True` if **at least one** condition is `True`.
- `not` changes `True` to `False` and `False` to `True`.
- Logical operators are commonly used with `if`, `elif`, `else`, and loops.

Membership Operators

Membership operators are used to check whether a value exists in a sequence or collection such as a **string**, **list**, **tuple**, **set**, or **dictionary**.

Python provides two membership operators:

- `in`
- `not in`

Membership Operators Table

Operator	Description	Example
`in`	Returns `True` if the value is present in the sequence	`"Python" in text`
`not in`	Returns `True` if the value is not present in the sequence	`"Java" not in text`

Example

In [ ]:
```
fruits = ["Apple", "Banana", "Mango"]

print("Apple" in fruits)
print("Orange" in fruits)
print("Orange" not in fruits)
```

Output

```
True
False
True
```

Example with String

In [ ]:
```
text = "Python Programming"

print("Python" in text)
print("Java" in text)
```

Output

```
True
False
```

Key Points

- Membership operators check whether a value exists in a sequence or collection.
- `in` returns `True` if the value is found.

- `not in` returns `True` if the value is not found.
- They work with strings, lists, tuples, sets, and dictionaries.
- Membership operators are commonly used with conditional statements and loops.

Bitwise Operators

Bitwise operators perform operations on the **binary (bit)** representation of integers. They compare or manipulate individual bits of numbers.

Python provides the following bitwise operators:

- `&` (AND)
- `|` (OR)
- `^` (XOR)
- `~` (NOT)
- `<<` (Left Shift)
- `>>` (Right Shift)

Bitwise Operators Table

Operator	Name	Description	Example
`&`	Bitwise AND	Sets a bit to `1` if both bits are `1`	`5 & 3`
`|`	Bitwise OR	Sets a bit to `1` if either bit is `1`	`5 | 3`
`^`	Bitwise XOR	Sets a bit to `1` if the bits are different	`5 ^ 3`
`~`	Bitwise NOT	Inverts all the bits	`~5`
`<<`	Left Shift	Shifts bits to the left	`5 << 1`
`>>`	Right Shift	Shifts bits to the right	`5 >> 1`

Example

In [ ]:
```
a = 5
b = 3

print("AND :", a & b)
print("OR  :", a | b)
print("XOR :", a ^ b)
print("NOT :", ~a)
print("Left Shift :", a << 1)
print("Right Shift:", a >> 1)
```

Output

```
AND : 1
OR  : 7
```

```
XOR : 6
NOT : -6
Left Shift : 10
Right Shift: 2
```

Key Points

- Bitwise operators work with the binary representation of integers.
- They are mainly used in low-level programming, networking, encryption, and performance optimization.
- Beginners do not use bitwise operators as often as arithmetic or comparison operators.
- Understanding binary numbers makes bitwise operators easier to learn.

Operator Precedence

Operator precedence determines the order in which Python evaluates operators in an expression. Operators with **higher precedence** are evaluated before operators with **lower precedence**.

You can use **parentheses** `()` to change the order of evaluation.

Operator Precedence Table

Precedence	Operators	Description
Highest	`()`	Parentheses
2	`**`	Exponentiation
3	`+x` , `-x` , `~x`	Unary Operators
4	`*` , `/` , `//` , `%`	Multiplication, Division, Floor Division, Modulus
5	`+` , `-`	Addition, Subtraction
6	`<<` , `>>`	Bitwise Shift
7	`&`	Bitwise AND
8	`^`	Bitwise XOR
9	`|`	Bitwise OR
10	`==` , `!=` , `>` , `<` , `>=` , `<=`	Comparison
11	`not`	Logical NOT
12	`and`	Logical AND
Lowest	`or`	Logical OR

Example 1

In [ ]:
```
result = 10 + 5 * 2
print(result)
```

Output

```
20
```

Explanation

Multiplication is performed before addition.

Example 2

result = (10 + 5) * 2

print(result)

Output

```
30
```

Explanation

The expression inside the parentheses is evaluated first.

Key Points

- Python follows a fixed order when evaluating expressions.
- Operators with higher precedence are evaluated before lower-precedence operators.
- Use parentheses `()` to make expressions easier to read and control the order of evaluation.

Using Parentheses or Round Bracket or ()

Parentheses `()` are used to control the order in which expressions are evaluated. Expressions inside parentheses are evaluated **before** other operators.

Using parentheses makes code easier to read and helps avoid mistakes.

Example 1

```
result = 10 + 5 * 2

print(result)
```

Output

```
20
```

Explanation

Multiplication is performed before addition.

```
10 + (5 × 2)
= 10 + 10
= 20
```

Example 2

```
result = (10 + 5) * 2

print(result)
```

Output

```
30
```

Explanation

The expression inside the parentheses is evaluated first.

```
(10 + 5) × 2
= 15 × 2
= 30
```

Example 3

```
result = (8 - 2) * (4 + 1)

print(result)
```

Output

```
30
```

Explanation

Both expressions inside the parentheses are evaluated first.

```
(8 - 2) = 6
(4 + 1) = 5

6 × 5 = 30
```

Why Use Parentheses?

- Control the order of operations.

- Improve code readability.
- Avoid unexpected results.
- Make complex expressions easier to understand.

Key Points

- Expressions inside `()` are evaluated first.
- Parentheses override the default operator precedence.
- Use parentheses whenever an expression may be confusing.
- Well-placed parentheses make code more readable and maintainable.

Common Errors

Beginners often make mistakes while using operators. Understanding these errors will help you write correct and efficient Python programs.

---

1. Using `=` Instead of `==`

The `=` operator is used for **assignment**, while `==` is used for **comparison**.

Incorrect

```
a = 10

if a = 10:
    print("Equal")
```

Output

```
SyntaxError: invalid syntax
```

Correct

```
a = 10

if a == 10:
    print("Equal")
```

---

2. Division by Zero

A number cannot be divided by zero.

Example

```
a = 10
b = 0
```

```
print(a / b)
```

Output

```
ZeroDivisionError: division by zero
```

---

3. Mixing Incompatible Data Types

Arithmetic operators cannot be used directly with incompatible data types.

Example

```
print("10" + 5)
```

Output

```
TypeError: can only concatenate str (not "int") to str
```

---

4. Forgetting Parentheses

Incorrect use of parentheses may produce unexpected results.

Example

```
result = 10 + 5 * 2

print(result)
```

Output

```
20
```

If you want addition first, use:

```
result = (10 + 5) * 2
```

---

5. Using the Wrong Operator

Using an incorrect operator can produce unexpected results.

Example

```
print(2 ^ 3)
```

`^` is the **Bitwise XOR** operator, **not** the power operator.

Use:

```
print(2 ** 3)
```

Output

```
8
```

Tips to Avoid Errors

- Use `==` for comparison and `=` for assignment.
- Never divide a number by zero.
- Ensure operands have compatible data types.
- Use parentheses to improve readability and control evaluation order.
- Read error messages carefully to identify and fix mistakes.

Best Practices

Following good programming practices makes your code easier to read, understand, and maintain.

Best Practices for Using Operators

- Use parentheses `()` to make expressions clear and avoid confusion.
- Use meaningful variable names instead of single letters whenever possible.
- Use `==` for comparison and `=` for assignment.
- Avoid dividing a number by zero.
- Ensure operands have compatible data types before performing operations.
- Keep expressions simple and easy to understand.
- Add spaces around operators to improve readability.
- Test your code with different input values.
- Read error messages carefully and fix issues step by step.
- Write clean and well-formatted code.

Good Example

```
length = 10
width = 5

area = length * width

print(area)
```

Avoid This

```
a=10
b=5
c=a*b
print(c)
```

Although this code works, the first example is easier to read because it uses meaningful variable names and proper spacing.

Key Points

- Write simple and readable code.
- Use meaningful variable names.
- Follow proper spacing around operators.
- Use parentheses when needed.
- Test your programs regularly to ensure correct results.

Simple Programs (10)

The following programs demonstrate how different Python operators work.

Program 1: Add Two Numbers

Write a Python program to add two numbers and display the result.

Program 2: Find the Area of a Rectangle

Write a Python program to calculate the area of a rectangle using the multiplication operator.

Program 3: Check Whether Two Numbers Are Equal

Write a Python program to compare two numbers using the `==` operator.

Program 4: Check the Larger Number

Write a Python program to determine whether one number is greater than another using the `>` operator.

Program 5: Check Voting Eligibility

Write a Python program to check whether a person's age is greater than or equal to 18.

Program 6: Use the Logical `and` Operator

Write a Python program to check whether a number is greater than 10 and less than 50.

Program 7: Check Membership in a List

Write a Python program to check whether `"Python"` exists in a list of programming languages.

Program 8: Calculate the Square of a Number

Write a Python program to calculate the square of a number using the exponentiation operator (`**`).

Program 9: Find the Remainder

Write a Python program to find the remainder when one number is divided by another.

Program 10: Perform Floor Division

Write a Python program to perform floor division on two numbers and display the result.

Practice Programs (10)

Solve the following programs on your own before checking the solutions.

Program 1

Write a Python program to subtract two numbers.

Program 2

Write a Python program to multiply three numbers.

Program 3

Write a Python program to divide two numbers and display the quotient.

Program 4

Write a Python program to calculate the cube of a number using the exponentiation operator.

Program 5

Write a Python program to check whether two numbers are not equal.

Program 6

Write a Python program to check whether a number lies between 1 and 100 using logical operators.

Program 7

Write a Python program to check whether `"apple"` is present in a list of fruits.

Program 8

Write a Python program to use the assignment operator (`+=`) to increase a variable by 20.

Program 9

Write a Python program to perform a left shift operation on a number by one position.

Program 10

Write a Python program to calculate the value of the following expression:

```
(15 + 5) * 2 - 10 // 2
```

Display the result.

Cheat Sheet

Arithmetic Operators

Operator	Description	Example
`+`	Addition	`10 + 5`
`-`	Subtraction	`10 - 5`
`*`	Multiplication	`10 * 5`
`/`	Division	`10 / 5`
`%`	Modulus	`10 % 3`
`**`	Exponentiation	`2 ** 3`
`//`	Floor Division	`10 // 3`

Assignment Operators

Operator	Example	Equivalent To
`=`	`x = 10`	Assign value
`+=`	`x += 5`	`x = x + 5`
`-=`	`x -= 5`	`x = x - 5`
`*=`	`x *= 5`	`x = x * 5`
`/=`	`x /= 5`	`x = x / 5`
`%=`	`x %= 5`	`x = x % 5`
`**=`	`x **= 2`	`x = x ** 2`
`//=`	`x //= 5`	`x = x // 5`

Comparison Operators

Operator	Description
`==`	Equal to
`!=`	Not equal to
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to

Logical Operators

Operator	Description
`and`	Returns `True` if both conditions are `True`
`or`	Returns `True` if at least one condition is `True`
`not`	Reverses the result of a condition

Identity Operators

Operator	Description
`is`	Returns `True` if both variables refer to the same object
`is not`	Returns `True` if both variables refer to different objects

---

Membership Operators

Operator	Description
`in`	Returns `True` if a value exists in a sequence
`not in`	Returns `True` if a value does not exist in a sequence

Bitwise Operators

Operator	Description
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`~`	Bitwise NOT
`<<`	Left Shift
`>>`	Right Shift

Operator Precedence (Highest → Lowest)

1. `()`
2. `**`
3. `+x`, `-x`, `~x`
4. `*`, `/`, `//`, `%`
5. `+`, `-`
6. `<<`, `>>`
7. `&`
8. `^`
9. `|`
10. `==`, `!=`, `>`, `<`, `>=`, `<=`
11. `not`
12. `and`
13. `or`

Quick Tips

- Use `=` to assign a value.
- Use `==` to compare values.
- Use `()` to control the order of operations.
- `and` requires all conditions to be `True`.
- `or` requires at least one condition to be `True`.
- `not` reverses a Boolean value.
- `in` checks whether a value exists in a sequence.
- `//` removes the decimal part of a division result.
- `%` returns the remainder after division.
- `**` calculates the power of a number.

Summary

In this lesson, you learned about **Python Operators**, which are used to perform different operations on data.

In this lesson, you learned:

- What operators are and why they are used.
- The different types of Python operators:
  - Arithmetic Operators
  - Assignment Operators
  - Comparison Operators
  - Logical Operators
  - Identity Operators
  - Membership Operators
  - Bitwise Operators
- How to perform mathematical calculations using arithmetic operators.
- How to assign and update values using assignment operators.
- How to compare values using comparison operators.
- How to combine conditions using logical operators.
- How to check object identity using identity operators.
- How to check whether a value exists in a sequence using membership operators.
- The basics of bitwise operators.
- How operator precedence affects the evaluation of expressions.
- How parentheses `()` can be used to control the order of operations.
- Common operator-related errors and best coding practices.

Key Takeaways

- Operators perform operations on variables and values.
- Different operators are used for different purposes.

- Comparison and logical operators always return `True` or `False`.
- Parentheses improve readability and control the order of evaluation.
- Write clean, readable expressions and use meaningful variable names.
- Practice using operators to become familiar with their behavior.

What's Next?

In **Part 4**, you will learn about **Conditional Statements**, including:

- `if` Statement
- `if...else` Statement
- `if...elif...else` Statement
- Nested `if` Statements
- Short-Hand `if` and `if...else`
- Real-world examples and practice programs

These concepts will help you make decisions in your Python programs based on different conditions.

Thank You! 🚀